PORTFOLIO · BUSINESS ANALYTICS CASE STUDY

From static reports to self-service reliability insight

A STAR walkthrough of the outage dashboard I built as a Business Analytics Intern at PG&E

Wasim Khan

Power BI · PySpark · Palantir Foundry

OVERVIEW

The story in four parts

S

Situation

Engineers waited on hand-pulled data requests

T

Task

A dashboard engineers could use on their own

A

Action

Pipeline, measures, self-service design, validation

R

Result

Most questions now self-served; requests only for edge cases

Then: the skills from this project that carry over to any analytics role.

SITUATION

Every question became a data request

Engineers ask

Reliability engineers need SAIDI, SAIFI and CAIDI by region and time period

They file a request

Each question goes to our team as a data request

We pull it by hand

Answers take time, and similar questions keep coming back

The bottleneck: engineers handle outages one by one, but the big-picture numbers waited in a queue.

SITUATION · THE METRICS

How utility reliability is measured

SAIDI

duration

$$\frac{\text{Customer minutes out}}{\text{Customers served}}$$

Outage minutes per customer served

SAIFI

frequency

$$\frac{\text{Customers interrupted}}{\text{Customers served}}$$

Interruptions per customer served

CAIDI

restoration

$$\frac{\text{Customer minutes out}}{\text{Customers interrupted}}$$

Minutes to restore, per outage

TASK · THE SPEC IN CODE

One calculation engineers could trust

Coverage
All three metrics for All, Planned and Unplanned outages

Any slice
Any region and period, plus YTD and cumulative

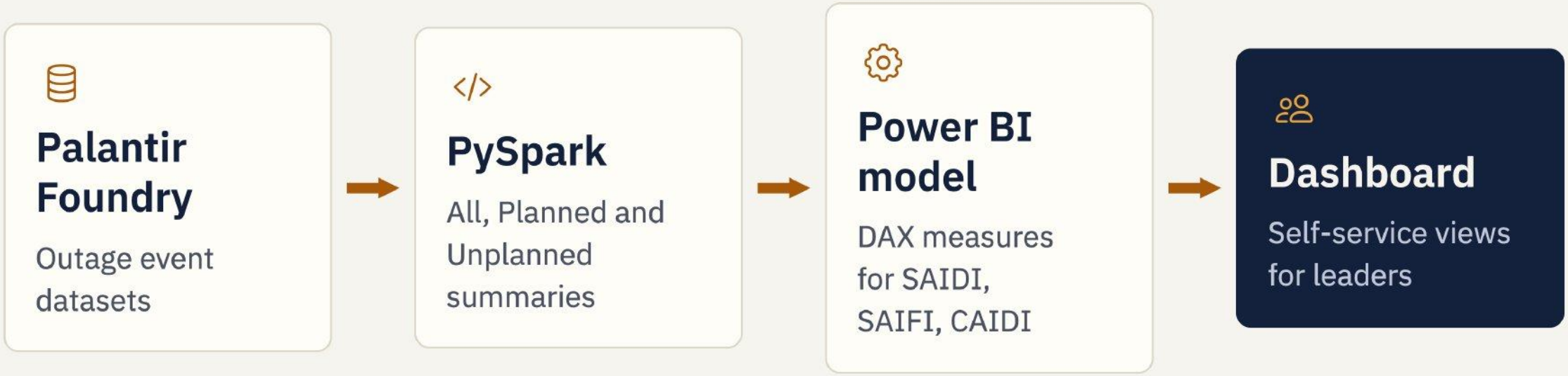
Self-service
Answers without filing a data request

Trust
Every number matches the source-level answer key

```
summary = (outages
  .filter(~F.col("major_event_day"))
  .groupBy("region", "month", "outage_type")
  .agg(
    F.sum(F.col("cust") * F.col("mins"))
      .alias("cust_min"),  # minutes out
    F.sum("cust").alias("cust_int"))  # interrupted
  .join(served, ["region", "month"])  # served
  .withColumn("SAIDI", F.col("cust_min") / F.col("served"))
  .withColumn("SAIFI", F.col("cust_int") / F.col("served"))
  .withColumn("CAIDI", F.col("cust_min") / F.col("cust_int")))
```

ACTION 1 · THE PIPELINE

From source data to screen



Also investigated: semantic-model refresh limits between Foundry and Power BI, so the team knew when the reporting data was current.

ACTION 2 · THE MEASURES

Measures that track the year

Monthly

How a single month performed

Year-to-date

January through the selected month, this year

Cumulative

Running total of completed months, compared with prior years

DAX PATTERN, SIMPLIFIED

```
SAIDI YTD =
DIVIDE(
  CALCULATE(
    SUM(Outages[CustMinutes]),
    DATESYTD('Date'[Date])
  ),
  [Customers Served]
)
```

Same shape for SAIFI and CAIDI, sliced by month, year, region and cause

ACTION 3 · THE DESIGN

Designed for self-service

Showing: Unplanned · 2025 YTD · Region → Cause

Region / Cause	SAIDI	SAIFI
▾ Region A	xx.x	x.xx
Equipment	xx.x	x.xx
Vegetation	xx.x	x.xx
Weather	xx.x	x.xx
▸ Region B	xx.x	x.xx
▸ Region C	xx.x	x.xx
Grand total	xx.x	x.xx

Schematic, not real data. One grand total, no redundant subtotals.

FIELD PARAMETER

A slicer that picks which **metrics** the table shows, so one page replaces three

✓ SAIDI ✓ SAIFI CAIDI

Drill-down: expand a region to see its causes

Multi-select filters: compare regions or causes at once

Filter-summary card: the dark bar always says what you're looking at

S T A R

RESULT

From waiting in a queue to looking it up

BEFORE

File a data request → wait for a manual pull → ask again for the next slice

AFTER

Open the dashboard → pick region and period → answer on the spot

The vast majority of questions are now answered in the dashboard.

Data requests remain for niche edge cases, so our team’s time goes to the analysis that needs it.

TAKEAWAYS

What carries over to any analytics role

What I did at PG&E	The transferable skill
Defined SAIDI, SAIFI and CAIDI clearly	Defining ratio KPIs so they add up correctly
Drilled from region down to cause	Root-cause analysis with drill-down
Foundry → PySpark → Power BI	Building the pipeline from source to dashboard
Reconciled totals to the source	Validating every number before it ships
Replaced manual data pulls	Self-service reports people explore on their own